САМОСТОЯТЕЛЬНАЯ РАБОТА СТУДЕНТА (СРО)

Вариант 14. Журнал событий веб-сервиса

Технология: Apache Cassandra

Дисциплина: «Проектирование и применение NoSQL-технологий»
3 курс, образовательная программа «Информационные системы»

Отчёт по мини-проекту
2026

1. Цель и постановка задачи

Цель работы — спроектировать и реализовать работающий прототип NoSQL-подсистемы журналирования событий веб-сервисов на базе Apache Cassandra. Система должна обеспечивать высокую скорость записи, эффективную выборку последних событий, фильтрацию по уровню критичности и контроль размера партиций (защита от hot partition) с помощью техники time-bucketing.

Предметная область: микросервисная архитектура (auth, payment, order, notification, api-gateway), каждый сервис генерирует поток логов (DEBUG / INFO / WARN / ERROR / FATAL). Требуется хранить историю, быстро получать «последние N» и отчёты по ошибкам за день.

2. Требования

2.1 Функциональные требования (≥ 6)

- FR1. Регистрация веб-сервиса (имя, описание, владелец).
- FR2. Создание (запись) события с полями: service, level, message, source, user_id, request_id, extra.
- FR3. Получение последних N событий сервиса за указанный день (newest first).
- FR4. Выборка событий заданного уровня (ERROR, WARN, …) за день.
- FR5. Диапазонный запрос по времени, в том числе охватывающий несколько day-buckets.
- FR6. Удаление конкретного события по полному первичному ключу.
- FR7. Подсчёт количества событий в day-партиции.
- FR8. Дополнительная запись в hourly-bucket для high-volume сервисов.

2.2 Нефункциональные требования (≥ 3)

- NFR1. Размер каждой партиции ограничен одним днём (или часом) — bounded partitions.
- NFR2. Latency чтения «последних N» должна быть предсказуемой (clustering order DESC + LIMIT).
- NFR3. Автоматическое истечение данных (TTL 30 дней для daily, 7 дней для hourly).
- NFR4. Горизонтальная масштабируемость за счёт добавления узлов Cassandra.

3. Предметная область и Access Patterns

Модель данных Cassandra проектируется «от запросов». Ниже перечислены ≥ 8 Access Patterns с приоритетом и ожидаемым результатом.

#	Access Pattern	Приоритет	Ожидаемый результат
AP1	Последние N событий сервиса за день	Высокий	≤ N строк, newest first
AP2	События уровня L сервиса за день	Высокий	Список ERROR/WARN/…
AP3	События в интервале [t1,t2] (кросс-день)	Средний	Объединённый список
AP4	Последние N из hourly-bucket	Средний	Демо anti-hot-partition
AP5	COUNT событий в day-partition	Низкий	Число строк
AP6	Список зарегистрированных сервисов	Низкий	Метаданные
AP7	Удаление события	Средний	Отсутствие в SELECT
AP8	Upsert message события	Низкий	Новое значение при чтении

4. Сравнение альтернатив и ADR

Рассматривались MongoDB, Redis, PostgreSQL/TimescaleDB, Elasticsearch и Apache Cassandra. Выбрана Cassandra по следующим причинам:

• Write-heavy нагрузка и линейное горизонтальное масштабирование.
• Composite partition key (service, day) даёт bounded partitions и защиту от hot partition.
• CLUSTERING ORDER BY event_time DESC делает «последние N» тривиальным LIMIT.
• TimeWindowCompactionStrategy оптимален для time-series.
• TTL на уровне таблицы без ручного удаления.

MongoDB проигрывает в предсказуемости размера партиций; Redis дорог по RAM для длительного хранения; PostgreSQL сложнее горизонтально шардировать; Elasticsearch избыточен (full-text не требуется). Подробный ADR — в файле docs/ADR.md.

5. Схема данных и архитектура

Основные таблицы (фрагмент CQL):

```
CREATE TABLE events_by_service_day (
  service_name text, day text,
  event_time timestamp, event_id timeuuid,
  level text, message text, source text,
  user_id text, request_id text, extra map&lt;text,text&gt;,
  PRIMARY KEY ((service_name, day), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC, event_id DESC)
  AND compaction = {'class': 'TimeWindowCompactionStrategy', ...}
  AND default_time_to_live = 2592000;
```

Дополнительно: events_by_service_hour (hourly bucket для high-volume), events_by_level (денормализация под AP2), services (метаданные). Архитектурная схема — docs/architecture.svg / architecture.md.

6. Реализация и фрагменты кода

Прототип на Python 3: модули config, db, models, repository, schema. Репозиторий реализует CREATE / READ / UPDATE (upsert) / DELETE. Скрипт scripts/seed.py генерирует ≥ 60 синтетических событий за последние 3 дня. Демонстрация запросов — scripts/demo_queries.py (≥ 8 запросов, 2 нетривиальных).

```
# Создание события
event = Event(service_name="auth-service", level="ERROR",
              message="Invalid JWT", source="auth.controller")
repo.create(event)

# AP1: последние 10
rows = repo.read_latest("auth-service", "2026-10-14", limit=10)

# AP3 (non-trivial): кросс-дневной диапазон
rows = repo.read_time_range("order-service", start, end)
```

7. Запросы и результаты

После seed выполняются демо-запросы Q1–Q10. Среди них два нетривиальных:

• Q5 — события order-service за последние 48 ч: приложение перебирает day-buckets и сливает результаты, сортируя глобально по event_time DESC.
• Q8 — демонстрация контроля размера партиций: для api-gateway выводятся COUNTS по трём дням; видно, что каждая партиция ограничена одним днём, в отличие от unbounded partition без bucketing.

8. Тестирование

Файл tests/test_repository.py содержит ≥ 3 сценария (pytest):

1. test_create_and_read_latest — запись 5 событий и проверка порядка newest-first.
2. test_read_by_level — фильтр ERROR vs INFO.
3. test_delete_and_count — удаление и проверка COUNT / отсутствия в SELECT.
4. (bonus) test_time_range_cross_day — кросс-дневной диапазон.

Тесты автоматически пропускаются, если Cassandra недоступна (pytest.mark.skipif). Запуск: pytest tests/ -v.

9. Ограничения

• Один узел, SimpleStrategy RF=1 — достаточно для учебного прототипа.
• Кросс-дневные запросы последовательные (можно заменить на execute_concurrent).
• Нет authZ/authN на уровне приложения, нет real-time streaming (Kafka и т.п.).
• Синтетические данные; реальные PII / секреты запрещены правилами СРО.

10. Выводы

Реализован полный прототип журнала событий на Cassandra, удовлетворяющий всем требованиям варианта 14 и общим критериям СРО: ≥ 6 FR + ≥ 3 NFR, ≥ 8 Access Patterns, ADR с обоснованием выбора, работающий CRUD-код, ≥ 30 (фактически 60) синтетических записей, ≥ 8 демо-запросов (2 нетривиальных), ≥ 3 теста, GitHub-структура, README, отчёт и презентация. Техника daily/hourly bucketing успешно демонстрирует контроль размера партиций и борьбу с hot partition — ключевые темы лекции по Cassandra.

Ссылка на GitHub-репозиторий: см. README.md (после публикации).